Universidad Nacional de Río Cuarto

Sistemas Operativos

(Código 1965)

Taller 1: Arquitectura RISC-V

Kernel mínimo en M-mode y S-mode con manejo de traps

Departamento de Computación
Facultad de Ciencias Exactas, Físico-Químicas y Naturales

Alumno
Tomás Rodeghiero

Fecha de entrega: 19 de abril de 2026

Índice

1. Objetivo del trabajo **2**

2. Entorno de desarrollo **2**

2.1. Herramientas utilizadas 2

2.2. Estructura de la entrega 2

3. Parte 1 - Hello World en M-mode **3**

3.1. Descripción 3

3.2. Compilación y ejecución 3

3.3. Salida obtenida 3

3.4. Captura de pantalla mostrando el kernel corriendo y el código fuente del archivo kernel.c 3

3.5. Código fuente 4

4. Parte 2 - S-mode y manejo de traps **5**

4.1. Contexto 5

4.2. Cómo se invoca `trap(cause)` desde `start.s` 5

4.3. Ejecución base 5

4.4. Prueba comentando la primera línea de `kernel_main()` 6

4.5. Respuesta conceptual (paso 6) 6

4.6. Implementación del TO DO en `trap()` 6

4.7. Código fuente final del archivo kernel.c 7

5. Conclusiones **9**

Objetivo del trabajo

Este taller tuvo como objetivo entender, con una práctica concreta, cómo arranca y se ejecuta un kernel mínimo en RISC-V bajo distintos niveles de privilegio. La idea no fue solo compilar y correr ejemplos, sino mirar qué pasa en cada etapa del boot y por qué.

El trabajo se organizó en dos partes complementarias:

- **Parte 1:** validación del arranque básico en *Machine Mode* (M-mode), compilación del proyecto y salida por UART.
- **Parte 2:** transición a *Supervisor Mode* (S-mode), análisis del mecanismo de traps y resolución del `TO DO` dentro de `trap()`.

En términos de aprendizaje, el foco estuvo en relacionar el código con el comportamiento real del procesador: qué instrucciones son legales en cada modo, cómo se reporta una excepción y de qué manera se puede diagnosticar desde el kernel.

Entorno de desarrollo

2.1 Herramientas utilizadas

- Emulador: `qemu-system-riscv32` (QEMU 10.2.2).
- Toolchain: `riscv64-unknown-elf-gcc` (GCC 15.1.0).
- Sistema de compilación: `make` con `PREFIX=riscv64-unknown-elf-`.

2.2 Estructura de la entrega

- `00-hello-world/`: código y pruebas de la Parte 1.
- `01-hello-world-S-mode/`: código y pruebas de la Parte 2.
- `README.md`: guía general del taller.
- `entrega/`: carpeta con los archivos finales de la entrega.
- `informe-taller1-riscv.pdf`: informe final en PDF.

Parte 1 - Hello World en M-mode

3.1 Descripción

En este primer paso se trabaja con un kernel muy simple que inicia en M-mode y escribe por consola serie usando UART mapeada en memoria. Es el punto de partida para comprobar que la herramienta de compilación, el linker script y la ejecución sobre QEMU están funcionando como se espera.

3.2 Compilación y ejecución

Listing 3.1: Comandos utilizados en la Parte 1

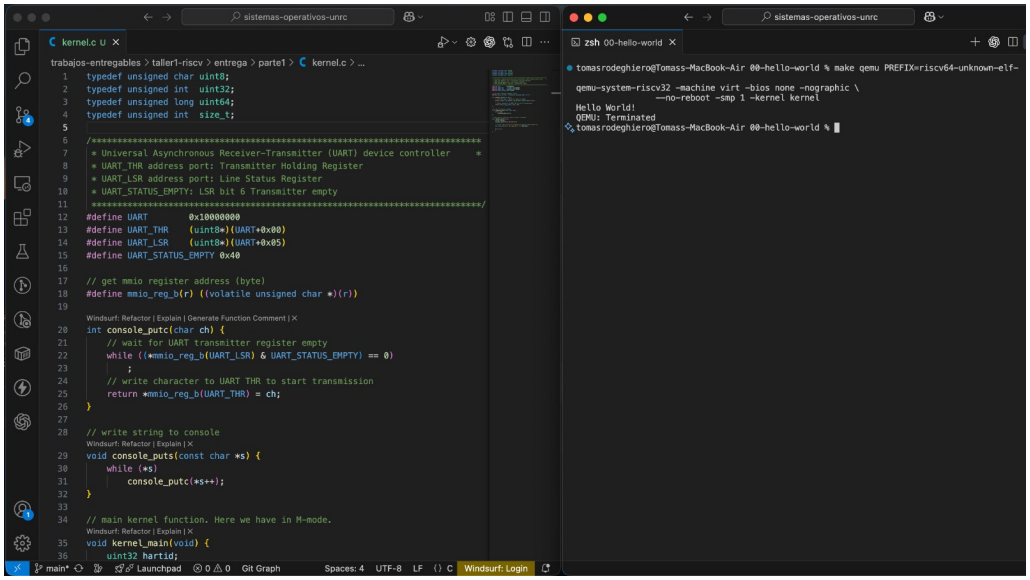
```
cd 00-hello-world
make PREFIX=riscv64-unknown-elf-
make qemu PREFIX=riscv64-unknown-elf-
```

3.3 Salida obtenida

```
Hello World!
```

La salida observada fue la esperada: `Hello World!`.

3.4 Captura de pantalla mostrando el kernel corriendo y el código fuente del archivo kernel.c



3.5 Código fuente

kernel.c - Parte 1

```c
typedef unsigned char uint8;
typedef unsigned int  uint32;
typedef unsigned long uint64;
typedef unsigned int  size_t;

/*************************************************************************
 * Universal Asynchronous Receiver-Transmitter (UART) device controller     *
 * UART_THR address port: Transmitter Holding Register
 * UART_LSR address port: Line Status Register
 * UART_STATUS_EMPTY: LSR bit 6 Transmitter empty
 *************************************************************************/
#define UART        0x10000000
#define UART_THR    (uint8*)(UART+0x00)
#define UART_LSR    (uint8*)(UART+0x05)
#define UART_STATUS_EMPTY 0x40

// get mmio register address (byte)
#define mmio_reg_b(r) ((volatile unsigned char *)(r))

int console_putc(char ch) {
    // wait for UART transmitter register empty
    while ((*mmio_reg_b(UART_LSR) & UART_STATUS_EMPTY) == 0)
        ;
    // write character to UART THR to start transmission
    return *mmio_reg_b(UART_THR) = ch;
}

// write string to console
void console_puts(const char *s) {
    while (*s)
        console_putc(*s++);
}

// main kernel function. Here we have in M-mode.
void kernel_main(void) {
    uint32 hartid;
    console_puts("Hello World!\n");

    // to do: verify we are in M-mode by reading some m-mode CSR
    asm volatile("csrr %0, mhartid": "=r"(hartid));

    for (;;) {
    }
}
```

Parte 2 - S-mode y manejo de traps

4.1 Contexto

En esta parte el cambio importante es que el kernel deja de ejecutarse en M-mode y pasa a S-mode durante el boot. Ese detalle modifica qué instrucciones son válidas y, por lo tanto, vuelve relevante el manejo de excepciones.

Además, el proyecto originalmente ya trae armado un esquema de traps en dos niveles:

1. **Una capa en ensamblador (`start.s`)**, donde la rutina `s_trap` captura el evento.
2. **Una capa en C (`kernel.c`)**, donde la función `trap(cause)` interpreta la causa y muestra información útil por UART.

Este diseño separa bien responsabilidades: el ensamblador hace la captura mínima de bajo nivel y C se encarga de la lógica de diagnóstico.

4.2 Cómo se invoca `trap(cause)` desde `start.s`

Invocación desde `s_trap`

```
s_trap:
    csrr a0, scause
    call trap
```

La invocación es directa y sigue la convención de llamada de RISC-V:

- `csrr a0, scause` lee el CSR `scause` y guarda su valor en `a0`.
- En la ABI de RISC-V, `a0` es el registro del primer argumento.
- Luego, `call trap` transfiere el control a `trap(uint cause)` usando ese valor como parámetro `cause`.

En otras palabras, el handler en ensamblador no interpreta la causa: solo la toma del hardware y se la pasa a C para su procesamiento.

4.3 Ejecución base

```
trap!!!
```

4.4 Prueba comentando la primera línea de `kernel_main()`

Se comentó temporalmente:

```
// asm volatile("csrr a0, mhartid");
```

Resultado obtenido:

```
Hello World from supervisor mode!
```

4.5 Respuesta conceptual (paso 6)

La excepción del paso 4 al ejecutar el código aparece porque `kernel_main()` ya está corriendo en S-mode, pero la primera línea intenta leer `mhartid`, que es un CSR privilegiado asociado al modo máquina.

Desde S-mode esa lectura no está permitida. Por eso, cuando la CPU intenta ejecutar esa instrucción, la arquitectura la marca como instrucción ilegal y dispara un trap. El valor `scause = 2` coincide justamente con *Illegal Instruction*, y eso explica por qué al comentar esa línea el problema desaparece.

4.6 Implementación del TO DO en `trap()`

Se implementó:

- Separación entre interrupción y excepción usando el bit más significativo de `scause`.
- Obtención del código de causa en los bits bajos.
- Detección explícita de *Illegal Instruction* (código 2).
- Mensajes de diagnóstico por UART para verificar el comportamiento en ejecución.

Salida final con implementación:

```
trap!!!
trap type: exception
exception code: 2
exception detail: Illegal Instruction
```

4.7 Código fuente final del archivo kernel.c

`kernel.c` final (Parte 2)

```c
typedef unsigned char uint8;
typedef unsigned int  uint32;
typedef unsigned int  uint;
typedef unsigned long uint64;
typedef unsigned int  size_t;

/*****************************************************************************
 * Universal Asynchronous Receiver-Transmitter (UART) device controller     *
 * UART_THR address port: Transmitter Holding Register
 * UART_LSR address port: Line Status Register
 * UART_STATUS_EMPTY: LSR bit 6 Transmitter empty
 *****************************************************************************/
#define UART        0x10000000
#define UART_THR    (uint8*)(UART+0x00)
#define UART_LSR    (uint8*)(UART+0x05)
#define UART_STATUS_EMPTY 0x40
#define SCAUSE_INTERRUPT_MASK (1U << 31)
#define SCAUSE_CODE_MASK      0x7fffffffU
#define SCAUSE_ILLEGAL_INSTRUCTION 2U

// get mmio register address (byte)
#define mmio_reg_b(r) ((volatile unsigned char *)(r))

int console_putc(char ch) {
    // wait for UART transmitter register empty
    while ((*mmio_reg_b(UART_LSR) & UART_STATUS_EMPTY) == 0)
        ;
    // write character to UART THR to start transmission
    return *mmio_reg_b(UART_THR) = ch;
}

// write string to console
void console_puts(const char *s) {
    while (*s)
        console_putc(*s++);
}

void console_putu(uint32 value) {
    char buf[11];
    int i = 0;

    if (value == 0) {
        console_putc('0');
        return;
    }

    while (value > 0) {
        buf[i++] = (char)('0' + (value % 10));
        value /= 10;
    }

    while (i > 0) {
```

```c
        console_putc(buf[--i]);
    }
}

// the trap handler (called from s_trap defined in start.s)
void trap(uint cause)
{
    console_puts("trap!!!\n");

    // Detect trap type and cause from scause:
    // bit 31: 1=interrupt, 0=exception
    // bits 30..0: exception/interrupt code
    uint is_interrupt = (cause & SCAUSE_INTERRUPT_MASK) != 0;
    uint code = cause & SCAUSE_CODE_MASK;

    if (is_interrupt) {
        console_puts("trap type: interrupt\n");
        console_puts("interrupt code: ");
        console_putu(code);
        console_putc('\n');
    } else {
        console_puts("trap type: exception\n");
        console_puts("exception code: ");
        console_putu(code);
        console_putc('\n');

        if (code == SCAUSE_ILLEGAL_INSTRUCTION) {
            console_puts("exception detail: Illegal Instruction\n");
        }
    }

    // In this step we stop execution after reporting the trap.
    for (;;) {
    }

}

const char *hello = "Hello World from supervisor mode!\n";

// main kernel function. Here we have in supervisor (kernel) mode.
void kernel_main(void) {
    asm volatile("csrr a0, mhartid");
    console_puts(hello);
    for (;;) {
    }
}
```

Conclusiones

Este taller me ayudó a entender, en un caso real y acotado, cómo se combinan niveles de privilegio, arranque del kernel y manejo de excepciones en RISC-V.

La Parte 1 me sirvió para validar la base del entorno y la salida por UART. La Parte 2 me agregó lo más interesante desde Sistemas Operativos: ver qué cambia al pasar a S-mode, detectar una excepción genuina por privilegios y resolver el handler para interpretar correctamente `scause`.